Materi Komunitas: Machine Learning

Pertemuan 6 - Aplikasi Basic Math: PCA dan Gradient Descent

1 Mengapa Kita Butuh PCA? (The "Why")

Sebelum kita membedah cara kerjanya, kita harus memahami mengapa PCA diciptakan. Dalam dunia data nyata, kita sering berhadapan dengan masalah yang disebut **Curse of Dimensionality** (Kutukan Dimensi Tinggi). Jika sebuah data memiliki terlalu banyak fitur (misalnya ratusan hingga ribuan kolom), algoritma Machine Learning akan kebingungan, proses komputasi menjadi sangat lambat, dan model rentan mengalami *overfitting*.

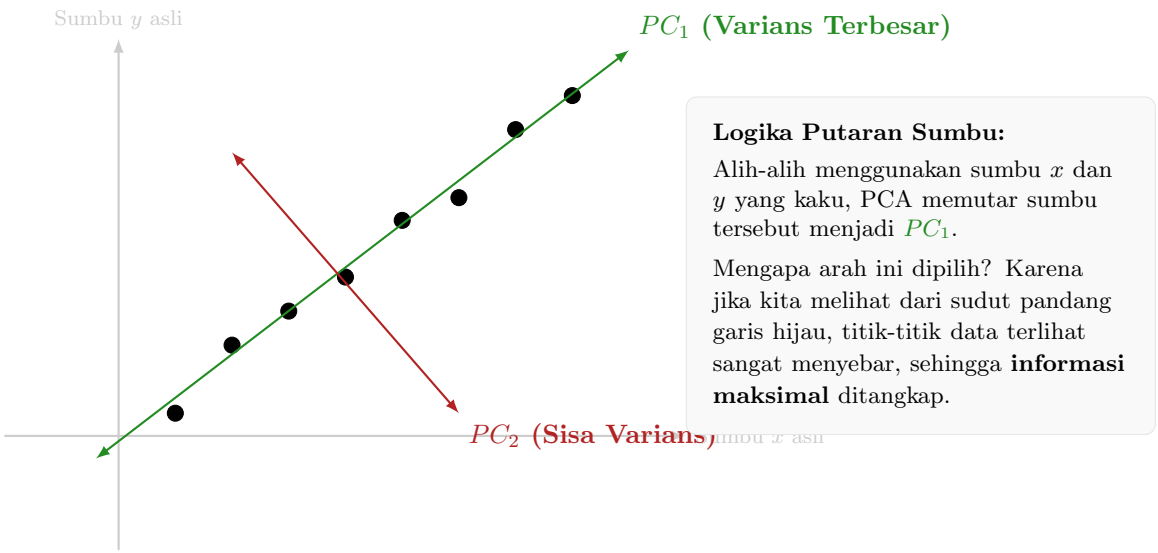
Tujuan Utama Penerapan PCA:

1. **Kompresi Data:** Mengecilkan ukuran dataset secara drastis namun tetap mempertahankan esensi informasinya.
2. **Akselerasi Model:** Mempercepat waktu *training* model Machine Learning karena beban fitur yang diproses menjadi jauh lebih sedikit.
3. **Visualisasi Data:** Memetakan data berdimensi tinggi (3D, 4D, atau lebih) ke dalam grafik 2D agar bisa dilihat dan dianalisis oleh mata manusia.

2 Geometris: Apa itu Varians?

Sebelum masuk ke istilah *Eigenvector*, kita harus memahami konsep **Varians**. Secara sederhana, varians adalah ukuran seberapa luas data kita menyebar. Dalam analisis data: **Data yang menyebar luas mengandung informasi lebih banyak daripada data yang menumpuk di satu titik.**

PCA bekerja dengan cara merotasi (memutar) sumbu koordinat asli (x, y) ke arah baru (PC_1, PC_2) di mana sumbu pertama menangkap rentang persebaran data yang paling panjang.



3 Komponen Utama: Eigenvalues & Eigenvectors

Bagaimana cara komputer memutar sumbu tersebut secara otomatis dan akurat? Di sinilah **Matematika Dasar** yang telah kita pelajari di minggu pertama berperan. Ini adalah "jantung" dari algoritma PCA.

Terdapat tiga komponen aljabar linear yang bekerja di balik layar:

Komponen	Peran dan Maknanya dalam PCA
Matriks Kovarians	Mengukur sejauh mana variabel dalam data berubah secara bersamaan (korelasi antar fitur). Matriks ini memetakan pola sebaran keseluruhan data.
Eigenvectors	Menentukan **Arah** dari sumbu baru (Principal Components). Sifat matematisnya memastikan bahwa antar garis sumbu selalu tegak lurus satu sama lain.
Eigenvalues	Menentukan **Besaran / Pentingnya** suatu arah. Semakin besar nilai Eigenvalue, semakin luas sebaran data di arah tersebut, yang berarti semakin banyak informasi yang dibawanya.

4 Langkah-Langkah Teknis (Step-by-Step)

Sekarang kita tahu apa itu PCA dan komponen pendukungnya. Lalu, bagaimana langkah-langkah nyata untuk melakukannya pada dataset? Berikut adalah algoritma PCA:

Langkah 1: Standardisasi Data (Mean Centering)
Langkah pertama yang **wajib** dilakukan adalah mencari rata-rata tiap fitur, lalu mengurangi setiap data dengan rata-ratanya agar titik pusat data berada di angka 0.

Langkah 2: Menghitung Matriks Kovarians
Mencari tahu korelasi antar semua fitur untuk melihat variabel mana yang bergerak searah atau berlawanan.

Langkah 3: Menghitung Eigenvectors & Eigenvalues
Dari Matriks Kovarians tersebut, kita hitung nilai *Eigenvectors* (arah sumbu baru) dan *Eigenvalues* (skor kekuatan varians di arah tersebut).

Langkah 4: Memilih Principal Components (PC)
Urutkan *Eigenvalue* dari yang terbesar ke terkecil. Kita akan menjadikan arah dengan nilai terbesar sebagai PC_1.

Langkah 5: Proyeksi (Transformasi Data)
Kalikan data asli dengan Eigenvector yang dipilih untuk mendapatkan koordinat baru yang dimensinya lebih ringkas.

Contoh Perhitungan Manual (Studi Kasus Sederhana)

Mari kita buktikan 5 langkah di atas menggunakan angka yang sangat sederhana. Misalkan kita memiliki 3 buah data dengan 2 fitur, yaitu X dan Y. Kita ingin mereduksi data 2D ini menjadi 1D saja.

Langkah 1: Data Asli & Standardisasi (Mean Centering)
Pertama, kita hitung Rata-rata (Mean) dari fitur X dan Y.

- Mean $X = \frac{1+2+3}{3} = \mathbf{2}$
- Mean $Y = \frac{2+4+3}{3} = \mathbf{3}$

Selanjutnya, kurangi setiap angka asli dengan rata-ratanya ($Data - Mean$).

Sampel	**Fitur** X	**Fitur** Y	$\rightarrow$	X **Centered** (x)	Y **Centered** (y)
1	1	2	$\rightarrow$	$1-2=\mathbf{-1}$	$2-3=\mathbf{-1}$
2	2	4	$\rightarrow$	$2-2=\mathbf{0}$	$4-3=\mathbf{1}$
3	3	3	$\rightarrow$	$3-2=\mathbf{1}$	$3-3=\mathbf{0}$

Langkah 2: Menghitung Matriks Kovarians (Σ)
Rumus kovarians untuk sampel (dibagi $n-1$, di mana $n=3$ sehingga $n-1=2$):

$$Var(X) = \frac{(-1)^2+0^2+1^2}{2} = \frac{2}{2} = \mathbf{1}$$

$$Var(Y) = \frac{(-1)^2+1^2+0^2}{2} = \frac{2}{2} = \mathbf{1}$$

$$Cov(X,Y) = \frac{(-1)(-1)+(0)(1)+(1)(0)}{2} = \frac{1}{2} = \mathbf{0.5}$$

Maka, Matriks Kovarians-nya adalah:

$$\Sigma = \begin{bmatrix} Var(X) & Cov(X,Y) \\ Cov(X,Y) & Var(Y) \end{bmatrix} = \begin{bmatrix} 1 & 0.5 \\ 0.5 & 1 \end{bmatrix}$$

Langkah 3: Menghitung Eigenvalues (λ)
Kita mencari nilai λ agar determinan dari $(\Sigma - \lambda I) = 0$.

$$\begin{aligned} \det \begin{pmatrix} 1-\lambda & 0.5 \\ 0.5 & 1-\lambda \end{pmatrix} &= 0 \\ (1-\lambda)(1-\lambda)-(0.5)(0.5) &= 0 \\ \lambda^2 - 2\lambda + 1 - 0.25 &= 0 \\ \lambda^2 - 2\lambda + 0.75 &= 0 \\ (\lambda-1.5)(\lambda-0.5) &= 0 \end{aligned}$$

Didapatkan dua nilai Eigenvalue: $\boldsymbol{\lambda_1 = 1.5}$ (Terbesar, akan jadi PC1) dan $\boldsymbol{\lambda_2 = 0.5}$ (Terkecil, akan dibuang).

Langkah 4: Menghitung Eigenvector untuk PC1

Masukkan nilai $\lambda_1 = 1.5$ kembali ke matriks untuk mencari arah vektor $\vec{v} = \begin{bmatrix} v_1 \\ v_2 \end{bmatrix}$:

$$\begin{bmatrix} 1-1.5 & 0.5 \\ 0.5 & 1-1.5 \end{bmatrix} \begin{bmatrix} v_1 \\ v_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

$$\begin{bmatrix} -0.5 & 0.5 \\ 0.5 & -0.5 \end{bmatrix} \begin{bmatrix} v_1 \\ v_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

Dari baris pertama, kita dapat persamaan: $-0.5v_1 + 0.5v_2 = 0 \Rightarrow \mathbf{v_1 = v_2}$.
Karena aturan vektor harus dinormalisasi (panjangnya 1, yaitu $\sqrt{v_1^2+v_2^2} = 1$), maka:

$$\sqrt{v_1^2+v_1^2} = 1 \implies \sqrt{2v_1^2} = 1 \implies v_1 = \frac{1}{\sqrt{2}} \approx \mathbf{0.707}$$

Jadi, Eigenvector untuk PC1 adalah $\begin{bmatrix} 0.707 \\ 0.707 \end{bmatrix}$.

Langkah 5: Proyeksi Data ke PC1
Kita kalikan data *Centered* kita dengan Eigenvector PC1 untuk mendapatkan koordinat 1D yang baru.

Hasil Akhir Reduksi Dimensi (Proyeksi)

Rumus: **Nilai PC1** $= (x_{centered} \times 0.707) + (y_{centered} \times 0.707)$

- **Sampel 1:** $(-1 \cdot 0.707) + (-1 \cdot 0.707) = \mathbf{-1.414}$
- **Sampel 2:** $(0 \cdot 0.707) + (1 \cdot 0.707) = \mathbf{0.707}$
- **Sampel 3:** $(1 \cdot 0.707) + (0 \cdot 0.707) = \mathbf{0.707}$

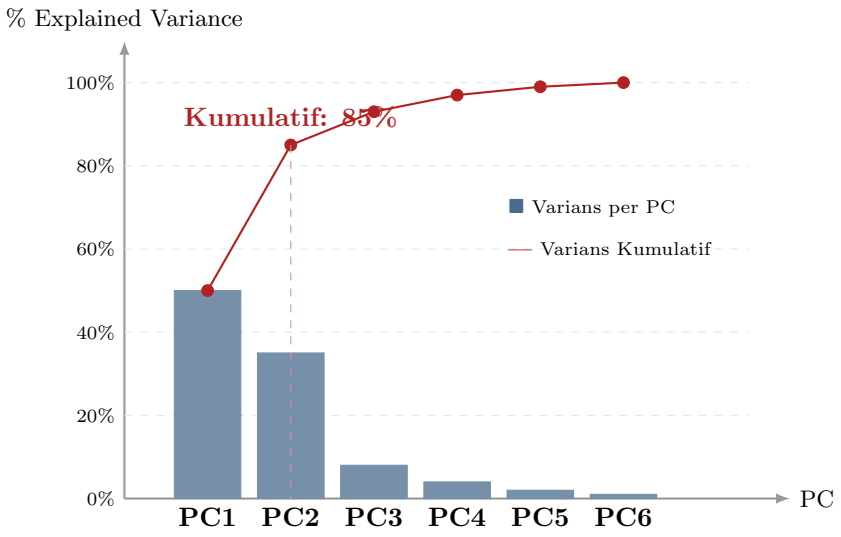
Selesai! Data kita yang awalnya memiliki 2 kolom (X dan Y) kini berhasil dikompresi menjadi 1 kolom saja (Nilai PC1), dengan mempertahankan 75% informasi aslinya.

5 Memilih Dimensi: Scree Plot & Explained Variance

Dalam aplikasi nyata, dataset kita mungkin memiliki 100 fitur. Berapa banyak Principal Component (PC) yang harus kita pertahankan?

Untuk mengetahuinya, kita menggunakan metrik **Explained Variance Ratio**. Pada perhitungan manual kita di atas, varians total adalah $\lambda_1 + \lambda_2 = 1.5 + 0.5 = 2.0$. Maka, PC1 menangkap $\frac{1.5}{2.0} = 75\%$ informasi, dan PC2 menangkap 25% informasi.

Dalam skala besar, kita memvisualisasikannya melalui **Scree Plot**.



Cara Membaca Scree Plot:

- **Balok Biru** menunjukkan porsi informasi individu. PC1 menampung paling banyak informasi, lalu ukurannya menurun drastis di PC berikutnya.
- **Garis Merah** adalah total informasi gabungan (Kumulatif). Jika kita menggabungkan PC1 (50%) dan PC2 (35%), kita sudah menyelamatkan **85% informasi asli**.
- **Kesimpulan:** Jika beberapa komponen pertama sudah mencakup **80% hingga 95%** informasi, maka sisa fiturnya bisa dibuang. Hasilnya, ukuran data menjadi jauh lebih kecil, namun model AI kita tetap cerdas!

Topik 2: Gradient Descent (Optimasi Model)

6 Mengapa Kita Butuh Gradient Descent?

Dalam Machine Learning, saat kita melatih sebuah model (misalnya Regresi Linear atau Neural Network), model tersebut awalnya akan memberikan prediksi yang tidak akurat. Selisih antara "Prediksi Model" dan "Data Aktual" disebut sebagai **Error** atau **Cost Function** (J).

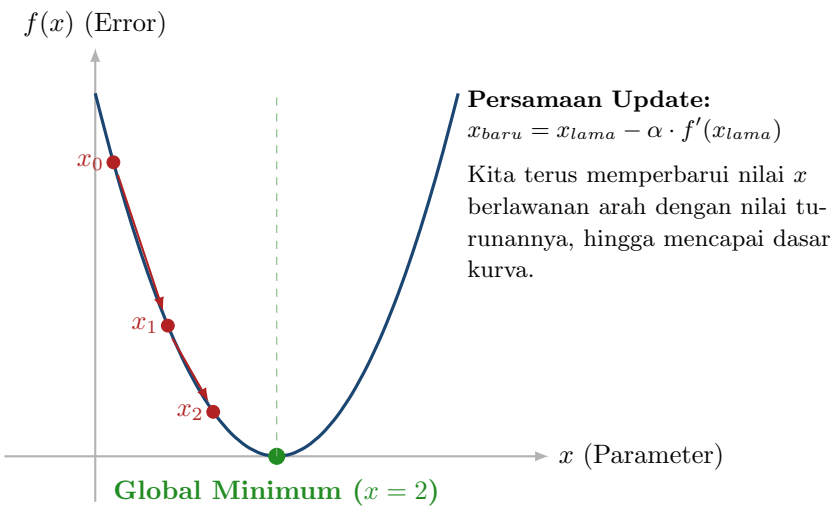
Tujuan utama dari fase *training* di Machine Learning adalah mencari kombinasi parameter (bobot/weight) yang menghasilkan nilai Error **paling minimum**. Untuk menemukan titik minimum ini secara efisien tanpa harus mencoba semua angka secara acak, kita menggunakan algoritma optimasi bernama **Gradient Descent**.

7 Konsep Matematika: Turunan & Learning Rate

Gradient Descent sangat bergantung pada konsep **Kalkulus Dasar**, secara spesifik adalah **Turunan (Derivative)**. Terdapat dua komponen utama dalam algoritma ini:

- **Gradient / Turunan ($f'(x)$):**
  Turunan dari sebuah fungsi menunjukkan tingkat kemiringan (gradien) pada titik tertentu. Secara matematis, vektor gradien selalu menunjuk ke arah *kenaikan* paling curam dari fungsi tersebut. Karena tujuan kita adalah mencari titik minimum (error terendah), maka kita harus bergerak ke **arah yang berlawanan dengan hasil turunan**.
- **Learning Rate (α):**
  Ini adalah *hyperparameter* yang menentukan **seberapa besar langkah** yang kita ambil pada setiap iterasi.
  - Jika α terlalu kecil: Konvergensi (pencarian titik minimum) akan berjalan sangat lambat.
  - Jika α terlalu besar: Langkah akan "melompati" titik minimum (*overshooting*) dan model gagal menemukan titik optimal.

8 Visualisasi dan Langkah Iterasi



9 Contoh Perhitungan Step-by-Step

Mari kita aplikasikan rumusnya. Misalkan kita memiliki fungsi error kuadratik sederhana:

$$f(x) = x^2 - 4x + 4$$

Langkah 1: Tentukan Turunan Pertama
Turunan dari fungsi di atas (gradien) adalah:

$$f'(x) = 2x - 4$$

Titik minimum teoritis (saat $f'(x) = 0$) berada pada $x = 2$. Mari kita lihat apakah Gradient Descent bisa menemukan angka 2 ini secara bertahap.

Langkah 2: Tentukan Parameter Awal

- Titik awal tebakan acak (*Initial value*): $\mathbf{x_0 = 0}$
- Learning rate: $\alpha = \mathbf{0.1}$

Proses Iterasi Gradient Descent

Iterasi 1:

- Hitung Gradien di titik x_0:
  $f'(0) = 2(0) - 4 = \mathbf{-4}$ *(Nilai negatif berarti kurva sedang turun ke kanan)*
- Update nilai x:
  $x_1 = x_0 - \alpha \cdot f'(x_0)$
  $x_1 = 0 - (0.1)(-4)$
  $x_1 = 0 + 0.4 = \mathbf{0.4}$

Iterasi 2:

- Hitung Gradien di titik x_1:
  $f'(0.4) = 2(0.4) - 4 = 0.8 - 4 = \mathbf{-3.2}$
- Update nilai x:
  $x_2 = x_1 - \alpha \cdot f'(x_1)$
  $x_2 = 0.4 - (0.1)(-3.2)$
  $x_2 = 0.4 + 0.32 = \mathbf{0.72}$

Iterasi 3:

- Hitung Gradien di titik x_2:
  $f'(0.72) = 2(0.72) - 4 = 1.44 - 4 = \mathbf{-2.56}$
- Update nilai x:
  $x_3 = x_2 - \alpha \cdot f'(x_2)$
  $x_3 = 0.72 - (0.1)(-2.56)$
  $x_3 = 0.72 + 0.256 = \mathbf{0.976}$

Analisis Hasil:
Perhatikan pergerakan nilai x: dari $\mathbf{0} \rightarrow \mathbf{0.4} \rightarrow \mathbf{0.72} \rightarrow \mathbf{0.976}$. Angka ini bergerak dengan pasti dan terus mendekati angka **2** (yang merupakan titik Global Minimum). Algoritma ini akan berhenti (konvergen) ketika hasil turunan $f'(x)$ sudah mendekati angka nol, yang berarti kita sudah berada di dasar kurva (error terendah).